AttnRes の depth-attention を教師とした Proxy-Block 動的深さルーティングの研究提案

研究提案資料

2026 年 4 月 22 日

要旨

本提案の目的は、Attention Residuals (AttnRes) が学習する *depth-attention* を、単なる residual mixer で終わらせず、因果的に使える *compute router* に変換することである。Attention Residuals (AttnRes) は過去層出力を深さ方向に softmax 集約するため表現学習上は魅力的だが、重み $\alpha_{i\to l}$ は source block の表現が実際に計算された後でないと正確には得られない。このため、Attention Residuals (AttnRes) の重みをそのまま online の block skip 判定に使うことは難しい。そこで本提案では、まず Block Attention Residuals (AttnRes) を teacher として学習し、各 block の入力表現から 予測 *summary* $\hat{b}_n$ と keep score r_n を出す軽量 predictor G_n を学習する。student 側では teacher と同じ pseudo-query w_m を用いて $\hat{b}_n$ を採点し、FULL 実行と PROXY 実行を切り替える。これにより、*residual mixing* と *compute routing* を同じ *depth-attention* 機構でつなぐことができる。新規性の中心は、任意の future layer routing そのものではなく、Attention Residuals (AttnRes) の pseudo-query / depth-attention を *proxy summary* を介して因果化し、*router* に昇格させる点に置く。

1. 背景と問題設定

標準的な PreNorm Transformer では、layer l の入力は

$$h_l = h_{l-1} + f_{l-1}(h_{l-1}) \tag{1}$$

と書ける。これを

$$v_0 := h_1, \qquad v_i := f_i(h_i) \tag{2}$$

と置けば、h_l は過去 layer 出力の単純和である。Attention Residuals (AttnRes) はこの固定加算を、深さ方向の softmax attention

$$h_l = \sum_{i<l} \alpha_{i\to l} v_i, \qquad \alpha_{i\to l} = \mathrm{softmax}_{i<l}\Big(w_l^\top \mathrm{RMSNorm}(v_i)\Big) \tag{3}$$

に置き換え、各層が過去表現を入力依存に選択できるようにする [1]。この発想は、残差接続の情報希釈を抑えつつ、深さ方向の選択性を与えるという意味で非常に魅力的である。

一方で、ここには計算削減上の壁がある。$\alpha_{i\to l}$ を正確に計算するには $v_i = f_i(h_i)$ が必要であり、これは source block i をすでに実行した後でないと得られない。したがって、Attention Residuals (AttnRes) をそのまま *online* の *compute router* とみなすことはできない。以前の議論で問題になっていたのはまさにこの点であり、*exact* な *depth-attention* は本質的に事後信号である。

そこで本提案の研究課題は次のように定める。

研究課題

RQ: Attention Residuals (AttnRes) が学習した depth-attention を教師信号として用い、*block* 実行前の *hidden state* だけから将来 usefulness を予測し、FULL 実行と PROXY 実行を切り替える causal な block router を構築できるか。

2. 関連研究の整理と本提案の位置づけ

関連研究は大きく 3 つの系統に分けて整理できる。

2.1. 残差接続・深さ方向集約の再設計

DenseFormer は current/past representations の weighted average を導入し、depth-weighted averaging によって情報流を改善した [2]。DeepCrossAttention は過去層の出力を input-dependent に再重み付けし、さらに depth-wise cross-attention を加えることで residual learning を強化した [3]。Attention Residuals (AttnRes) はその流れの最新版であり、過去層出力への深さ方向 softmax attention と block-level 実装を提案した [1]。この系列は *history* をどう混ぜるかを改善するが、通常は これから計算する *block* を先に止めるための router ではない。

2.2. 動的深さ・条件付き計算

Mixture-of-Depths (MoD) は layer ごとに top-k token routing を行い、self-attention と MLP に参加する token 数を制限する [4]。Router-Tuning は MoD 系 router だけを微調整することで、動的深さの導入コストを下げる [5]。LayerSkip は layer dropout と early-exit loss により早期終了を成立させる [6]。これらは計算削減に直接つながるが、*router* は通常、残差再設計とは別機構として設計される。

2.3. 任意 layer path の routing と decoding 上の制約

Radial Networks は token-level に次の layer を決める dynamic layer routing を扱い、層の skip や reuse まで許す [7]。このため、*future layer* を *top-1* で貪欲に選ぶという着想自体は既存研究に近い。したがって、本提案は「任意 future layer routing そのもの」を新規性の主張軸に置くべきではない。加えて、FFN-SkipLLM は early-exit / layer-dropping を generation に持ち込む際、KV cache の扱いが大きな障害になることを示し、FFN-only のスキップを提案した [8]。この知見は、本提案の初期実験を prefill 中心または FFN-first に設定すべきことを示唆する。

3. 提案手法: Proxy-Block AttnRes Router

3.1. 基本アイデア

本提案の要点は、Attention Residuals (AttnRes) の exact な重み $\alpha_{i\to l}$ を online で直接使うのではなく、*teacher* が学習した *depth-attention* を *student* に蒸留する点にある。teacher は Block Attention Residuals (AttnRes)、student は各 block の前に置かれた軽量 predictor G_n から構成される。

Table 1: 関連研究と本提案の差分

カテゴリ	代表例	強み	本提案との差分
残差再設計	DenseFormer [2], DCA [3], AttnRes [1]	depth 方向の情報再利用を改善	計算削減のための causal router を直接は与えない
動的深さ	MoD [4], Router-Tuning [5], LayerSkip [6]	compute を入力依存に削減できる	多くは別 router または early-exit 学習を前提
任意 layer routing	Radial Networks [7]	future layer path を動的に選べる	既に近い先行研究があるため、単純な next-hop routing だけでは弱い
Autoregressive-safe skipping	FFN-SkipLLM [8]	KV cache 問題を避けやすい	block 全体の routing ではなく FFN 中心
本提案	Proxy-Block AttnRes Router (PBAR)	AttnRes teacher の depth-attention を same pseudo-query で causal router に蒸留	新規性の中心は “proxy summary を介した因果化” と “residual mixing / compute routing の統合”

Teacher (Block AttnRes). block index を $n = 1, \dots, N$ とし、teacher の completed block summary を b_n^T と書く。teacher の前向き計算は

$$b_0^T = e(x), \tag{4}$$

$$h_n^T = \sum_{i<n} \alpha_{i\to n}^T b_i^T, \tag{5}$$

$$\alpha_{i\to n}^T = \mathrm{softmax}_{i<n}\left(w_n^\top \,\mathrm{RMSNorm}(b_i^T)\right), \tag{6}$$

$$b_n^T = F_n(h_n^T) \tag{7}$$

と書ける。ここで w_n は Attention Residuals (AttnRes) の pseudo-query である [1]。

teacher からは 2 種類の監督信号を抜く。

$$s_n^T[m] = w_m^\top \,\mathrm{RMSNorm}(b_n^T), \qquad m > n, \tag{8}$$

$$u_n^T = \frac{1}{N-n} \sum_{m>n} \alpha_{n\to m}^T. \tag{9}$$

$s_n^T[m]$ は block n が future block m からどう見られていたかの生スコア、u_n^T は将来平均 reuse 量である。

Student (proxy predictor + gate). 各 block の入力 h_n から、軽量 predictor G_n が

$$(\hat{b}_n, r_n) = G_n(h_n) \tag{10}$$

を出力する。$\hat{b}_n$ は “この block を本当に回した場合の summary の近似”、r_n は keep score である。推論時の hard gate は

$$z_n = \mathbf{1}[r_n > \theta] \tag{11}$$

とし、block memory に格納する summary を

$$\tilde{b}_n = z_n F_n(h_n) + (1 - z_n)\hat{b}_n \tag{12}$$

と定義する。つまり、$z_n = 1$ なら FULL 実行、$z_n = 0$ なら PROXY 実行である。

次 block の入力は、teacher と同じ pseudo-query を用いて

$$h_{n+1} = \sum_{i \le n} \tilde{\alpha}_{i \to n+1}\, \tilde{b}_i, \tag{13}$$

$$\tilde{\alpha}_{i \to n+1} = \mathrm{softmax}_{i \le n}\Big(w_{n+1}^{\top}\, \mathrm{RMSNorm}(\tilde{b}_i) \Big) \tag{14}$$

と作る。ここで重要なのは、*teacher* と *student* が同じ *pseudo-query* w_m を共有する点である。

3.2. なぜこの設計で “無理” を越えられるのか

以前に “AttnRes の depth-attention をそのまま compute router に転用するのは難しい” と整理したのは、true key b_n が block 実行後でないと得られないためである。この点は今も変わらない。違いは、student では true key の代わりに *predicted key* $\hat{b}_n$ を使うことだ。すると

$$\hat{s}_n[m] = w_m^{\top}\, \mathrm{RMSNorm}(\hat{b}_n) \tag{15}$$

を block 実行前に計算でき、exact な α ではなくても、teacher の future usefulness を 因果的に近似できる。つまり、無理だったのは *exact reuse* であり、本提案は *proxy* を介した *distillation* によってその壁を越える。

3.3. 学習損失

まず teacher を構築した後、student では backbone と pseudo-query を固定し、主に G_n のみを学習する。基本損失は次の 5 本でよい。

$$\mathcal{L}_{\mathrm{proxy}} = \sum_n \|\hat{b}_n - b_n^T\|_2^2, \tag{16}$$

$$\mathcal{L}_{\mathrm{depth}} = \sum_n \mathrm{KL}\Big(\mathrm{softmax}(s_n^T/\tau) \,\|\, \mathrm{softmax}(\hat{s}_n/\tau) \Big), \tag{17}$$

$$\mathcal{L}_{\mathrm{gate}} = \sum_n (r_n - u_n^T)^2, \tag{18}$$

$$\mathcal{L}_{\mathrm{logit}} = \mathrm{KL}\big(p_T(\cdot \mid x) \,\|\, p_S(\cdot \mid x)\big), \tag{19}$$

$$\mathcal{L}_{\mathrm{budget}} = \lambda \left(\frac{1}{N} \sum_n z_n - \rho \right)^2. \tag{20}$$

最終損失はこれらの重み付き和で与える。学習中は hard gate の代わりにシグモイド緩和を用い、推論時のみ hard 化する。

4. 新規性の置きどころ

本提案の新規性は、*future layer top-1 routing* そのものには置かない。そこは Radial Networks [7] に近く、単に “未来 block を貪欲に選ぶ” だけでは差別化が弱い。主張すべき点は次の 4 つである。

1. **AttnRes の depth-attention を post-hoc な解析信号で終わらせず、teacher signal として compute routing に昇格させること。**
2. **teacher / student の両方で same pseudo-query w_m を使い、residual mixing と compute routing を一つの depth-attention 機構でつなぐこと。**

3. **true summary ではなく predicted summary $\hat{b}_n$ を導入し、non-causal な重みを causal な routing signal に変換すること。**
4. **任意の長距離 jump を直接許すのではなく、block 順序を保ったまま FULL/PROXY を置換することで、retrofit しやすく安定な設計を与えること。**

この整理により、本提案は “AttnRes の応用” と “dynamic-depth router” の中間ではなく、*depth-attention* に基づく *unified residual/compute routing* として位置づけられる。

主張のしかた

強く主張する点: “AttnRes の pseudo-query を teacher / student の両側で共有し、predicted summary を介して causal router に蒸留する”。
主張を弱める点: “動的に次の層を選ぶこと自体が新しい”。この点は既存研究と重なるため、提案の芯に置かない。

5. 研究仮説

本提案では以下の 3 仮説を検証する。

- **H1:** teacher の depth-attention は、block 実行前の hidden state からでも proxy summary を介して十分に予測可能であり、一定割合の FULL block を削減できる。
- **H2:** same pseudo-query を再利用する student は、同一予算下で別 router をゼロから学習する方式よりも安定かつ data-efficient に学習できる。
- **H3:** arbitrary layer jump よりも、block 順序を保った FULL/PROXY 置換の方が、既存 backbone への retrofit と generation の安定性の両方で有利である。

6. 実験計画

6.1. 段階的な実験設計

Phase 1: Teacher の再現と微調整. まず Block Attention Residuals (AttnRes) を target task に合わせて fine-tune し、b_n^T, s_n^T, u_n^T を抽出する [1]。
Phase 2: Router 蒸留. backbone と pseudo-query を固定し、G_n のみを学習する。ここで予算率 $\rho \in \{0.25, 0.50, 0.75\}$ を振る。
Phase 3: 追加最適化. 必要であれば上位数 block だけ LoRA で再調整し、FULL/PROXY 混在下の分布ずれを補正する。

6.2. 評価指標

- **品質**: 言語モデル perplexity、タスク精度、logit KL、必要に応じて指示追従タスクの勝率。
- **効率**: 実行された FULL block 数 / token、理論 FLOPs、prefill tok/s、decode tok/s、VRAM 使用量。
- **蒸留の質**: $\mathcal{L}_{\text{depth}}$、teacher/student の routing 一致率、proxy summary 再構成誤差。

6.3. 比較対象

1. 通常の sequential baseline
2. Block Attention Residuals (AttnRes) teacher [1]
3. Mixture-of-Depths (MoD) / Router-Tuning 系 baseline [4,5]
4. LayerSkip [6]
5. Radial-like な separate router baseline [7]
6. decoding を含める場合は FFN-SkipLLM [8]

6.4. 重要な ablation

- same pseudo-query 再利用 vs separate router head
- block summary predictor のサイズ
- FULL/PROXY 2 択 vs FFN-only PROXY
- prefill のみ vs decode まで含む評価
- block 粒度 N の違い

6.5. 成功条件の一例

提案の成否を早く判断するため、まずは次の実務的な基準を置く。

- teacher 比で品質低下を小さく保ったまま、実行 FULL block 数を 20% 以上削減できること。
- 同一予算下で、separate router baseline より logit KL と品質の両方で優位であること。
- prefill 計測で tok/s が改善し、decode を含める場合でも KV cache に起因する大きな崩壊が起きないこと。

7. リスクと代替案

リスク 1: proxy summary の精度不足。 $\hat{b}_n$ が粗すぎると、routing は合っていても後続 block の入力分布が崩れる。対策として、まずは block 全体ではなく FFN-only PROXY を試し、必要なら $\mathcal{L}_{\text{proxy}}$ と $\mathcal{L}_{\text{logit}}$ を強める [8]。

リスク 2: 深い block の入力期待値とのずれ。 任意の長距離 jump は分布ずれが大きいため、本提案では block 順序を保持する FULL/PROXY 置換を採用する。これは arbitrary routing を直接導入するより安全である [7]。

リスク 3: decode 時の KV cache 問題。 early-exit や layer-dropping は autoregressive decoding で崩れやすいことが知られている [6,8]。そのため、初期段階では prefill 中心の評価を優先し、decode 拡張は FFN-only PROXY または cache 設計の追加検討を伴う第 2 段階に回す。

リスク 4: teacher 構築コスト. Block Attention Residuals (AttnRes) 自体の fine-tuning が高価な場合、LoRA や小規模 calibration set による teacher 構築から始める。Router-Tuning [5] と同様に、router 側だけを安価に学習する流れを活かす。

8. 期待される成果

本研究が成功した場合、成果は 3 層に分かれる。第一に、Attention Residuals (AttnRes) の depth-attention が *future compute* の必要量をどこまで予測できるかを定量化できる。第二に、same pseudo-query を shared mechanism として利用することで、残差再設計と動的深さを統合する新しい設計原理を提示できる。第三に、実装上は FULL/PROXY block 置換という retrofit しやすい手段を得られる。特に “単純な next-hop routing は既存研究が厚い” という状況を踏まえると、本提案の学術的価値は、*AttnRes* 由来の *depth-attention* を *causal router* に変換する理論と実装の両方を示す点にある。

9. まとめ

本提案は、Attention Residuals (AttnRes) の exact depth-attention をそのまま router に使うのではなく、teacher-student 蒸留と predicted summary を通じて 因果的に使える *block router* を作るものである。closest prior は Radial Networks [7] を中心とする dynamic layer routing だが、本提案の差別化点は、*residual mixing* と *compute routing* を *same pseudo-query* で統合すること、および *proxy summary* による因果化にある。研究の第一歩としては、prefill 中心・block 順序保持・FULL/PROXY 2 択から始めるのが最も堅い。

最終的な一文の売り

From depth attention to depth routing: AttnRes が学習した depth-attention を、predicted block summary と same pseudo-query によって causal な compute router へ蒸留する。

参考文献

[1] Kimi Team et al. *Attention Residuals*. arXiv:2603.15031, 2026. https://arxiv.org/abs/2603.15031

[2] Matteo Pagliardini, Amirkeivan Mohtashami, Francois Fleuret, Martin Jaggi. *DenseFormer: Enhancing Information Flow in Transformers via Depth Weighted Averaging*. arXiv:2402.02622, 2024. https://arxiv.org/abs/2402.02622

[3] Mike Heddes, Adel Javanmard, Kyriakos Axiotis, Gang Fu, MohammadHossein Bateni, Vahab Mirrokni. *DeepCrossAttention: Supercharging Transformer Residual Connections*. arXiv:2502.06785, 2025. https://arxiv.org/abs/2502.06785

[4] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, Adam Santoro. *Mixture-of-Depths: Dynamically allocating compute in transformer-based language models*. arXiv:2404.02258, 2024. https://arxiv.org/abs/2404.02258

[5] Shwai He, Tao Ge, Guoheng Sun, Bowei Tian, Xiaoyang Wang, Ang Li, Dong Yu. *Router-Tuning: A Simple and Effective Approach for Enabling Dynamic-Depth in Transformers*. arXiv:2410.13184, 2024. https://arxiv.org/abs/2410.13184

[6] Mostafa Elhoushi et al. *LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding*. arXiv:2404.16710, 2024. https://arxiv.org/abs/2404.16710

[7] Jordan Dotzel, Yash Akhauri, Ahmed S. AbouElhamayed, Carly Jiang, Mohamed Abdelfattah, Zhiru Zhang. *Radial Networks: Dynamic Layer Routing for High-Performance Large Language Models*. arXiv:2404.04900, 2024. https://arxiv.org/abs/2404.04900

[8] Ajay Jaiswal et al. *FFN-SkipLLM: A Hidden Gem for Autoregressive Decoding with Adaptive Feed Forward Skipping*. arXiv:2404.03865, 2024. https://arxiv.org/abs/2404.03865